



ScholarNest

Spark Azure Databricks

Databricks Spark Certification and beyond

Instructor: Prashant Kumar Pandey



Azure
Databricks



Absolute Beginner to Specialization in Apache Spark and Azure Databricks



Spark Dataframes Schema

Requirement:

You are given a flight time data set. This data set comes in a JSON file, and you are asked to load this Dataframe and create a Spark table. For your reference, here is one sample record, and here are details about the columns shown below.

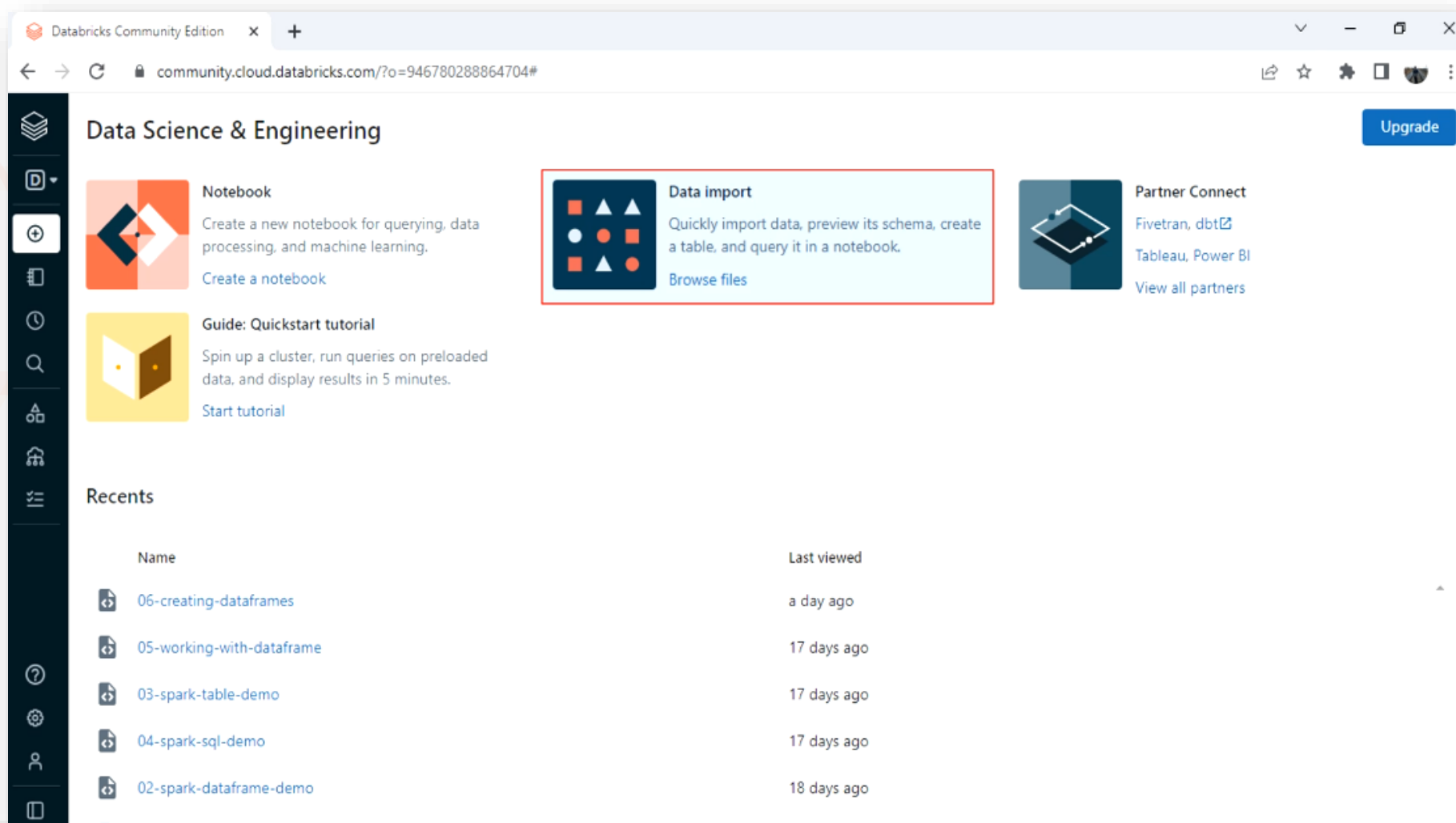
Flight Delay Data

```
{
  "FL_DATE": "1/1/2000",
  "OP_CARRIER": "DL",
  "OP_CARRIER_FL_NUM": 1451,
  "ORIGIN": "BOS",
  "ORIGIN_CITY_NAME": "Boston, MA",
  "DEST": "ATL",
  "DEST_CITY_NAME": "Atlanta, GA",
  "CRS_DEP_TIME": 1115,
  "DEP_TIME": 1113,
  "WHEELS_ON": 1343,
  "TAXI_IN": 5,
  "CRS_ARR_TIME": 1400,
  "ARR_TIME": 1348,
  "CANCELLED": 0,
  "DISTANCE": 946
}
```

1. Flight date in MM/DD/YYYY format
2. Unique carrier code
3. Flight number
4. Origin IATA Airport Code
5. Origin City Name
6. Destination IATA Airport Code
7. Destination City Name
8. Scheduled Departure time in HHMM
9. Actual Departure time in HHMM
10. Flight Duration in HHMM
11. TaxiIn taxi in time, in minutes
12. Scheduled Arrival time in HHMM
13. Actual Departure time in HHMM
14. Was it canceled? 1 for a true, and 0 is false
15. Distance in miles

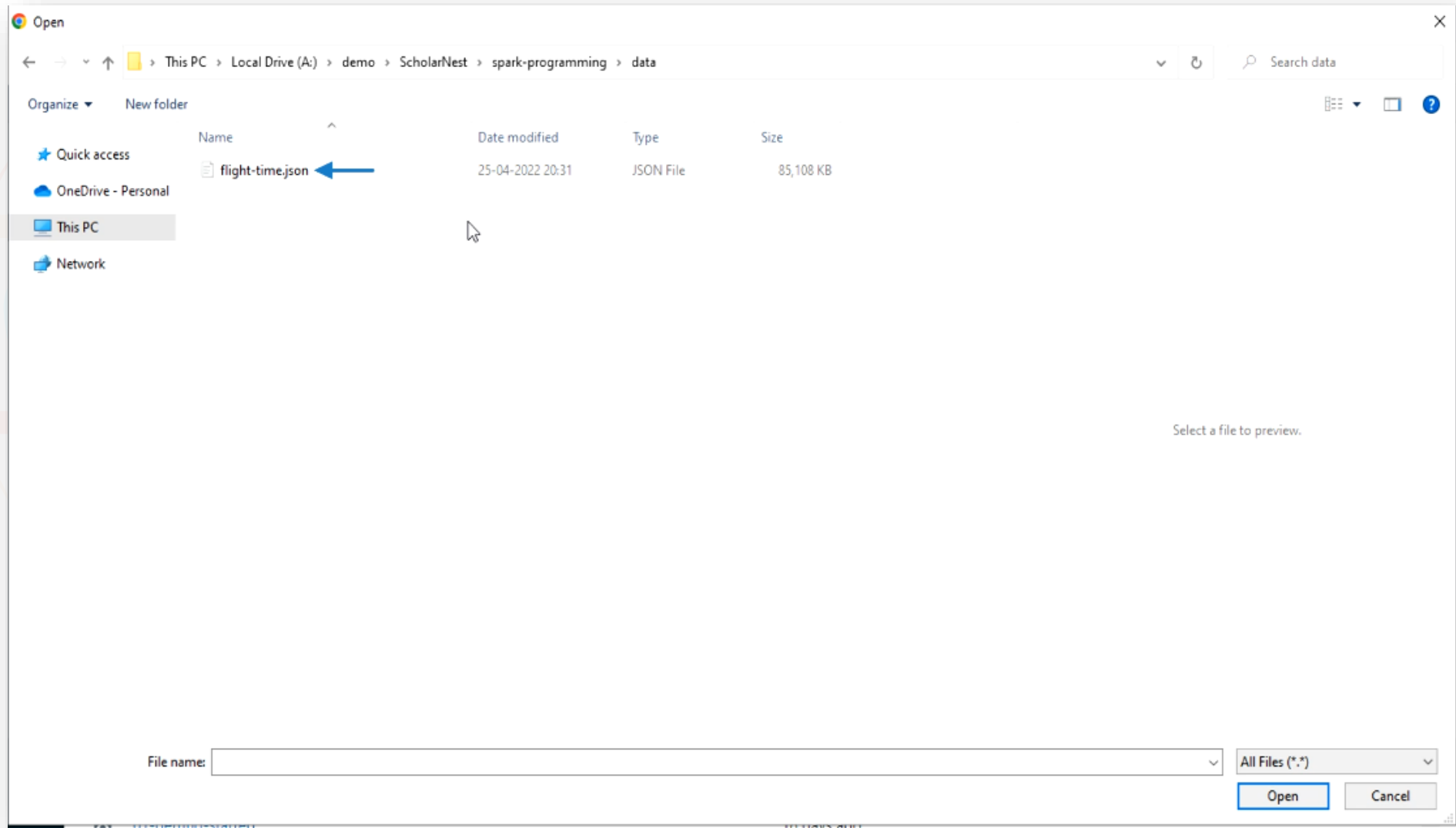
Load the table into a Dataframe:

Go to your Databricks workspace. My data file is available on my local machine. I will upload the data file to the cloud so I can work with it. Click this Data import button highlighted below.



Load the table into a Dataframe:

Navigate to the folder that contains the file and upload it.



Load the table into a Dataframe:

Your file is uploaded. It goes into the Databricks File System and sits in the `/FileStore/tables/` directory. The Databricks File System is popularly known as DBFS, and the community edition DBFS is built on top of the Amazon S3.

Create New Table

Data source ?

Upload File

S3

DBFS

Other Data Sources

Partner Integrations

DBFS Target Directory ?

/FileStore/tables/ (optional)

Select

Files ?

flight-time.json ✓

87.2 MB

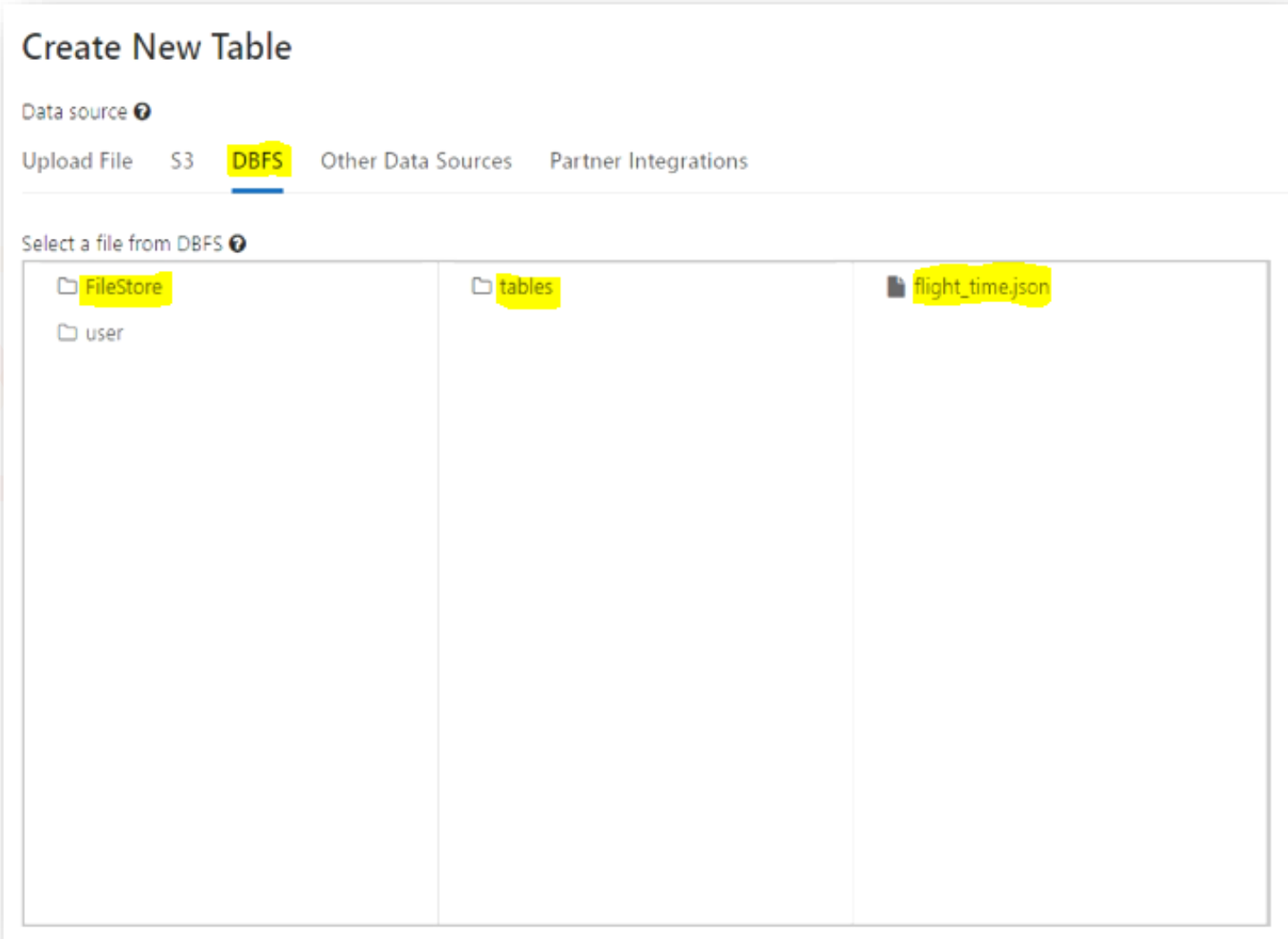
Remove file

✓File uploaded to `/FileStore/tables/flight_time.json`

Create Table with UI

Create Table in Notebook ?

To check your file, go to the DBFS tab and browse to the *FileStore/tables/*, and you will see the file. So my data file is now available in the cloud. Now you can create a new notebook to write code to meet the given requirement.



Create a new notebook. **(Reference - 07-dataframe-schema.ipynb)**

You can use the %fs ls command to check your data file, and we see the same in the output below.

The %fs ls is a DBFS command, and it is similar to the Linux ls command.

07-dataframe-schema

Python

demo-cluster

Cmd 1

1 %fs ls /FileStore/tables

	path	name	size	modificationTime
1	dbfs:/FileStore/tables/flight_time.json	flight_time.json	87150442	1650995258000

Showing all 1 rows.

Command took 5.95 seconds -- by prashant@scholarnest.com at 4/26/2022, 11:20:10 PM on demo-cluster

You have seen a similar code to create a Dataframe earlier.

I changed the format from CSV to JSON. Because this time, I am loading JSON data.

I also removed the header and inferSchema options. Because a JSON file doesn't come with a header row. The header option is only applicable for the CSV files.

The inferSchema option is not applicable for JSON. Because a JSON document comes with the column names in every record. Spark will always read the column names. In fact, it will read the column name with every record.

Cmd 2

```
1 flight_time_raw_df = spark.read \  
2     .format("json") \  
3     .load("/FileStore/tables/flight_time.json")
```

► (1) Spark Jobs

Command took 20.42 seconds -- by prashant@scholarnest.com at 4/26/2022, 11:22:50 PM on demo-cluster

Look at the schema of the data frame we created. You can see that Spark automatically inferred the schema for this Dataframe. Some columns are long, and others are string. And that is not correct.

07-dataframe-schema Python

demo-cluster (1) Spark Jobs

Command took 20.42 seconds -- by prashant@scholarnest.com at 4/26/2022, 11:22:50 PM on demo-cluster

Cmd 3

> 1 flight_time_raw_df.printSchema()

root
|-- ARR_TIME: long (nullable = true)
|-- CANCELLED: long (nullable = true)
|-- CRS_ARR_TIME: long (nullable = true)
|-- CRS_DEP_TIME: long (nullable = true)
|-- DEP_TIME: long (nullable = true)
|-- DEST: string (nullable = true)
|-- DEST_CITY_NAME: string (nullable = true)
|-- DISTANCE: long (nullable = true)
|-- FL_DATE: string (nullable = true)
|-- OP_CARRIER: string (nullable = true)
|-- OP_CARRIER_FL_NUM: long (nullable = true)
|-- ORIGIN: string (nullable = true)
|-- ORIGIN_CITY_NAME: string (nullable = true)
|-- TAXI_IN: long (nullable = true)
|-- WHEELS_ON: long (nullable = true)

Command took 0.13 seconds -- by prashant@scholarnest.com at 4/26/2022, 11:34:20 PM on demo-cluster

Schema inference is not reliable.

We have the following problems in schema inference:

1. Automatic inferring of schema is often incorrect
2. Inferring schema is additional work for Spark, and it takes some extra time
3. Schema inference is conflicting with the schema validation
4. It might also change the column order

Look at the schema of the data frame we created. You can see that Spark automatically inferred the schema for this Dataframe. Some columns are long, and others are string. And that is not correct.

07-dataframe-schema Python

demo-cluster (1) Spark Jobs

Command took 20.42 seconds -- by prashant@scholarnest.com at 4/26/2022, 11:22:50 PM on demo-cluster

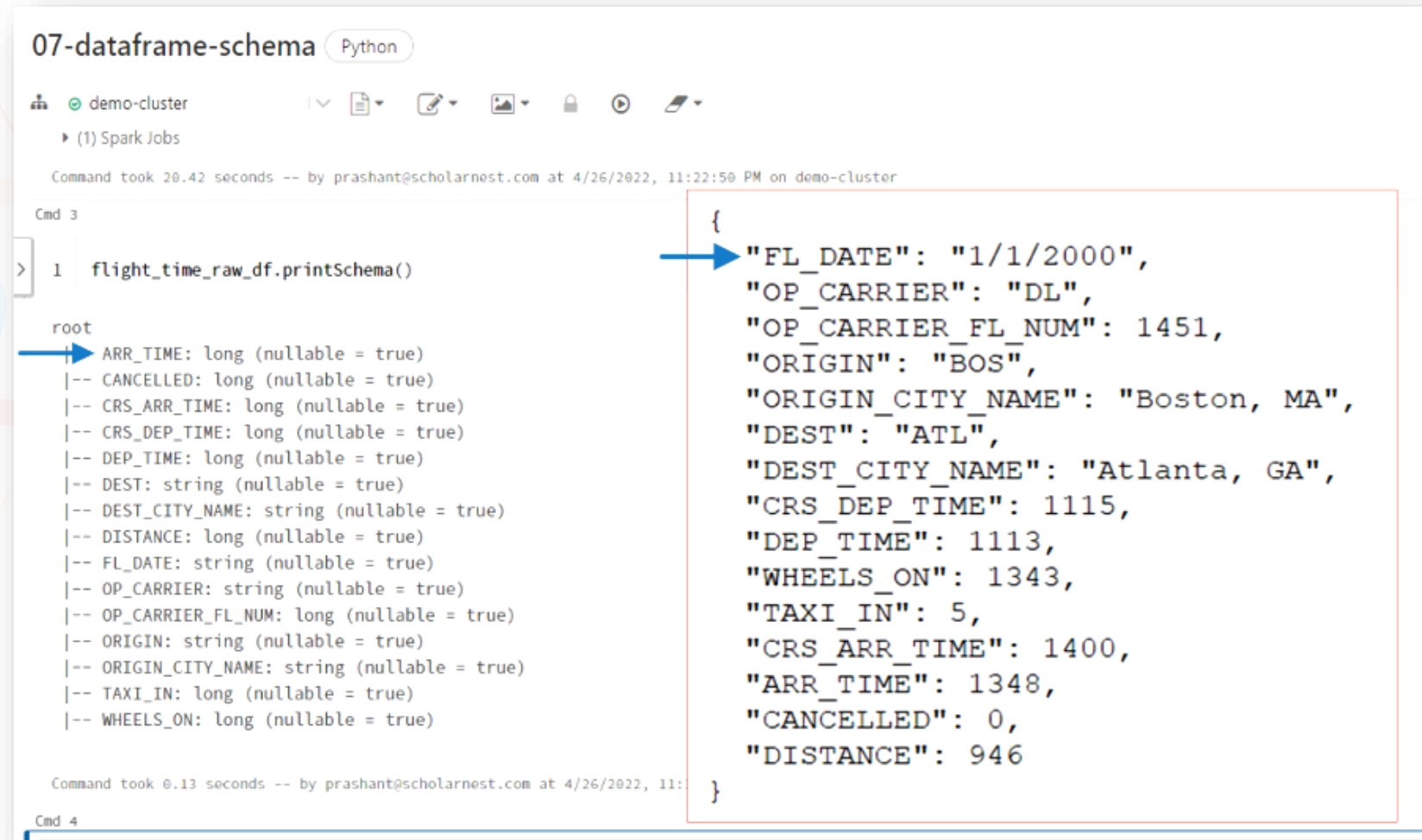
Cmd 3

> 1 flight_time_raw_df.printSchema()

root
|-- ARR_TIME: long (nullable = true)
|-- CANCELLED: long (nullable = true)
|-- CRS_ARR_TIME: long (nullable = true)
|-- CRS_DEP_TIME: long (nullable = true)
|-- DEP_TIME: long (nullable = true)
|-- DEST: string (nullable = true)
|-- DEST_CITY_NAME: string (nullable = true)
|-- DISTANCE: long (nullable = true)
|-- FL_DATE: string (nullable = true)
|-- OP_CARRIER: string (nullable = true)
|-- OP_CARRIER_FL_NUM: long (nullable = true)
|-- ORIGIN: string (nullable = true)
|-- ORIGIN_CITY_NAME: string (nullable = true)
|-- TAXI_IN: long (nullable = true)
|-- WHEELS_ON: long (nullable = true)

Command took 0.13 seconds -- by prashant@scholarnest.com at 4/26/2022, 11:34:20 PM on demo-cluster

Schema inference may also change the order of the columns. You can see it here if you compare the schema with your data file. The first column in Dataframe is ARR_TIME, whereas the first column in the file is FL_DATE. Column reordering is not a big problem. We should avoid creating a logic that depends on the column ordering but changing order often causes confusion. So we must avoid it.



```
07-dataframe-schema Python
demo-cluster
(1) Spark Jobs
Command took 20.42 seconds -- by prashant@scholarnest.com at 4/26/2022, 11:22:50 PM on demo-cluster

Cmd 3
> 1 flight_time_raw_df.printSchema()

root
  ARR_TIME: long (nullable = true)
  -- CANCELLED: long (nullable = true)
  -- CRS_ARR_TIME: long (nullable = true)
  -- CRS_DEP_TIME: long (nullable = true)
  -- DEP_TIME: long (nullable = true)
  -- DEST: string (nullable = true)
  -- DEST_CITY_NAME: string (nullable = true)
  -- DISTANCE: long (nullable = true)
  -- FL_DATE: string (nullable = true)
  -- OP_CARRIER: string (nullable = true)
  -- OP_CARRIER_FL_NUM: long (nullable = true)
  -- ORIGIN: string (nullable = true)
  -- ORIGIN_CITY_NAME: string (nullable = true)
  -- TAXI_IN: long (nullable = true)
  -- WHEELS_ON: long (nullable = true)

Command took 0.13 seconds -- by prashant@scholarnest.com at 4/26/2022, 11:22:50 PM on demo-cluster

Cmd 4
{
  "FL_DATE": "1/1/2000",
  "OP_CARRIER": "DL",
  "OP_CARRIER_FL_NUM": 1451,
  "ORIGIN": "BOS",
  "ORIGIN_CITY_NAME": "Boston, MA",
  "DEST": "ATL",
  "DEST_CITY_NAME": "Atlanta, GA",
  "CRS_DEP_TIME": 1115,
  "DEP_TIME": 1113,
  "WHEELS_ON": 1343,
  "TAXI_IN": 5,
  "CRS_ARR_TIME": 1400,
  "ARR_TIME": 1348,
  "CANCELLED": 0,
  "DISTANCE": 946
}
```

We have two steps to enforce a schema:

1. Create a schema definition for your data file.
2. Explicitly enforce the schema

The first step is to create a schema definition.

We have two approaches to do it.

1. Schema DDL String
2. StructType Object

Schema DDL string is the easiest method.
Here is my schema definition for the flight-time data. A schema DDL is a comma-separated string of column names and data types. So the first column name is FL_DATE, and it is a date. Similarly, the second column name is OP_CARRIER, and it is a string.
You can create a list of all the columns and their data types.

07-dataframe-schema

Python

demo-cluster

Cmd 1

1

%fs ls /FileStore/tables

	path	name	size	modificationTime
1	dbfs:/FileStore/tables/flight_time.json	flight_time.json	87150442	1650995258000

Showing all 1 rows.

Command took 5.95 seconds -- by prashant@scholarnest.com at 4/26/2022, 11:20:10 PM on demo-cluster

Cmd 2

1

flight_schema_ddl = """FL_DATE DATE, OP_CARRIER STRING, OP_CARRIER_FL_NUM INT, ORIGIN STRING,

2

ORIGIN_CITY_NAME STRING, DEST STRING, DEST_CITY_NAME STRING, CRS_DEP_TIME INT, DEP_TIME INT,

3

WHEELS_ON INT, TAXI_IN INT, CRS_ARR_TIME INT, ARR_TIME INT, CANCELLED BOOLEAN, DISTANCE INT"""

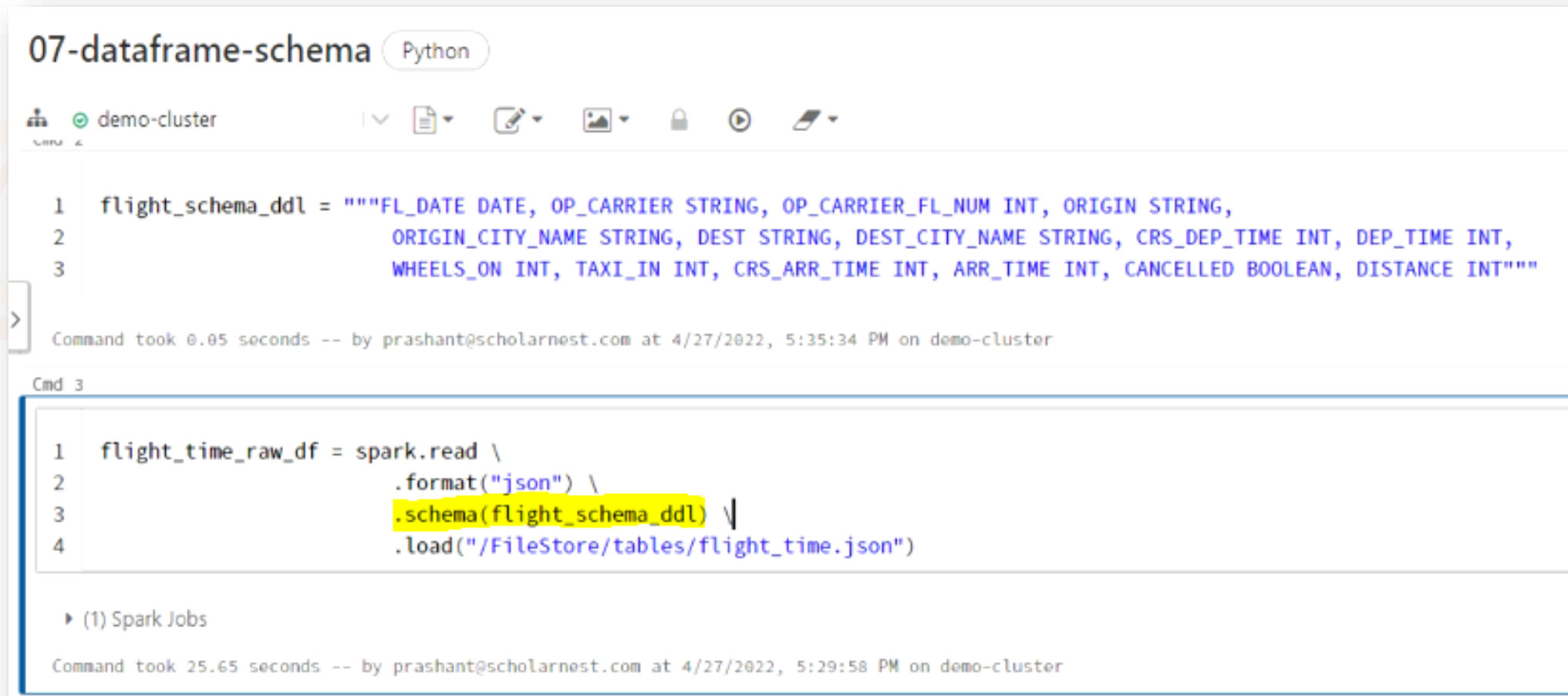
www.scholarnest.com

ScholarNest

© ScholarNest Technologies Pvt Ltd.

Once a schema is defined, you can enforce it to the DataframeReader by adding a schema method to your DataframeReader.

So here I am telling the DataframeReader to read a JSON format file using the flight_schema_ddl.



The screenshot shows a Databricks notebook interface. At the top, the notebook is titled "07-dataframe-schema" and the language is set to "Python". The toolbar includes icons for file operations, a lock, a play button, and a notebook icon. The main code area contains two code blocks. The first block defines a schema string named `flight_schema_ddl` with the following content: `FL_DATE DATE, OP_CARRIER STRING, OP_CARRIER_FL_NUM INT, ORIGIN STRING, ORIGIN_CITY_NAME STRING, DEST STRING, DEST_CITY_NAME STRING, CRS_DEP_TIME INT, DEP_TIME INT, WHEELS_ON INT, TAXI_IN INT, CRS_ARR_TIME INT, ARR_TIME INT, CANCELLED BOOLEAN, DISTANCE INT`. Below this code, a status message indicates the command took 0.05 seconds. The second code block, labeled "Cmd 3", shows the `spark.read` method being used to load a JSON file from the path `/FileStore/tables/flight_time.json`. The `.format("json")` and `.schema(flight_schema_ddl)` methods are highlighted in yellow. The final line of the second block is `.load("/FileStore/tables/flight_time.json")`. Below the code, a status message indicates the command took 25.65 seconds. At the bottom, there is a section for "Spark Jobs" showing "(1) Spark Jobs".

```
07-dataframe-schema Python

demo-cluster

1 flight_schema_ddl = """FL_DATE DATE, OP_CARRIER STRING, OP_CARRIER_FL_NUM INT, ORIGIN STRING,
2                       ORIGIN_CITY_NAME STRING, DEST STRING, DEST_CITY_NAME STRING, CRS_DEP_TIME INT, DEP_TIME INT,
3                       WHEELS_ON INT, TAXI_IN INT, CRS_ARR_TIME INT, ARR_TIME INT, CANCELLED BOOLEAN, DISTANCE INT"""

Command took 0.05 seconds -- by prashant@scholarnest.com at 4/27/2022, 5:35:34 PM on demo-cluster

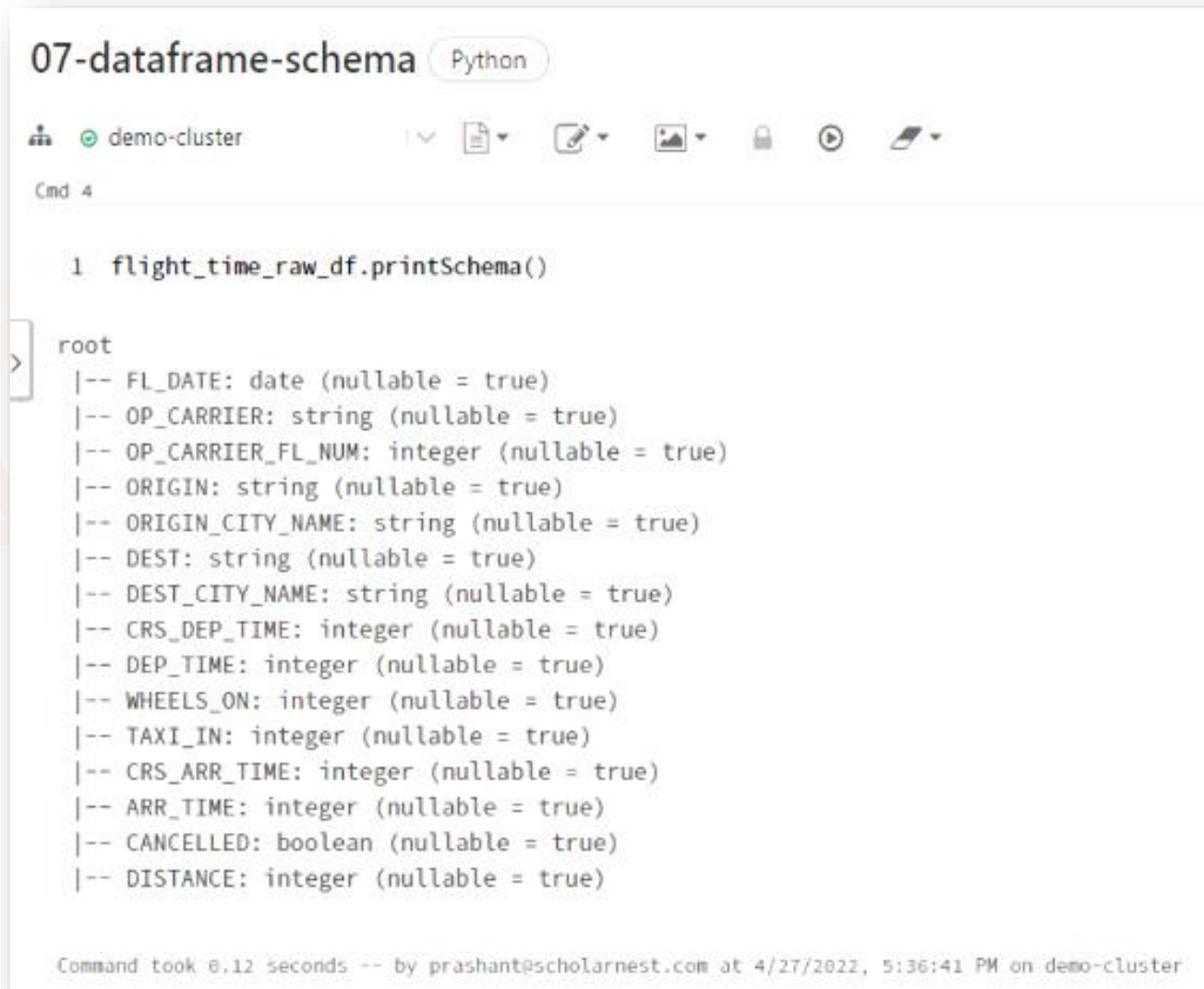
Cmd 3

1 flight_time_raw_df = spark.read \
2                       .format("json") \
3                       .schema(flight_schema_ddl) \
4                       .load("/FileStore/tables/flight_time.json")

(1) Spark Jobs

Command took 25.65 seconds -- by prashant@scholarnest.com at 4/27/2022, 5:29:58 PM on demo-cluster
```

You can re-check the schema.
The print schema is now showing it correctly.



```
07-dataframe-schema Python
demo-cluster
Cmd 4

1 flight_time_raw_df.printSchema()

root
 |-- FL_DATE: date (nullable = true)
 |-- OP_CARRIER: string (nullable = true)
 |-- OP_CARRIER_FL_NUM: integer (nullable = true)
 |-- ORIGIN: string (nullable = true)
 |-- ORIGIN_CITY_NAME: string (nullable = true)
 |-- DEST: string (nullable = true)
 |-- DEST_CITY_NAME: string (nullable = true)
 |-- CRS_DEP_TIME: integer (nullable = true)
 |-- DEP_TIME: integer (nullable = true)
 |-- WHEELS_ON: integer (nullable = true)
 |-- TAXI_IN: integer (nullable = true)
 |-- CRS_ARR_TIME: integer (nullable = true)
 |-- ARR_TIME: integer (nullable = true)
 |-- CANCELLED: boolean (nullable = true)
 |-- DISTANCE: integer (nullable = true)

Command took 0.12 seconds -- by prashant@scholarnest.com at 4/27/2022, 5:36:41 PM on demo-cluster
```

We can check the data once and see if we loaded it correctly. So here is my Dataframe shown below. It looks like FL_DATE is not read correctly, and hence it is NULL. I do see some nulls in between, but other rows have values.

07-dataframe-schema

Python

demo-cluster

1

-- ARR_TIME: integer (nullable = true)

-- CANCELLED: boolean (nullable = true)

-- DISTANCE: integer (nullable = true)

Command took 0.12 seconds -- by prashant@scholarnest.com at 4/27/2022, 5:36:41 PM on demo-cluster

Cmd 5

1 display(flight_time_raw_df)

(1) Spark Jobs

	FL_DATE	OP_CARRIER	OP_CARRIER_FL_NUM	ORIGIN	ORIGIN_CITY_NAME	DEST	DEST_CITY_NAME	CRS_DEP_TIME	DEP_TIME
1	null	DL	1451	BOS	Boston, MA	ATL	Atlanta, GA	1115	1113
2	null	DL	1479	BOS	Boston, MA	ATL	Atlanta, GA	1315	1311
3	null	DL	1857	BOS	Boston, MA	ATL	Atlanta, GA	1415	1414
4	null	DL	1997	BOS	Boston, MA	ATL	Atlanta, GA	1715	1720
5	null	DL	2065	BOS	Boston, MA	ATL	Atlanta, GA	2015	2010
6	null	US	2619	BOS	Boston, MA	ATL	Atlanta, GA	650	649
7	null	US	2621	BOS	Boston, MA	ATL	Atlanta, GA	1440	1446

Truncated results, showing first 1000 rows.

Click to re-execute with maximum result limits.

We can check the data once and see if we loaded it correctly. So here is my Dataframe shown below. It looks like FL_DATE is not read correctly, and hence it is NULL. I do see some nulls in between, but other rows have values. The CANCELLED column is also null everywhere.

07-dataframe-schema

Python

demo-cluster

1

-- ANN_TIME: integer (nullable = true)

-- CANCELLED: boolean (nullable = true)

-- DISTANCE: integer (nullable = true)

Command took 0.12 seconds -- by prashant@scholarnest.com at 4/27/2022, 5:36:41 PM on demo-cluster

Cmd 5

1 display(flight_time_raw_df)

(1) Spark Jobs

	FL_DATE	OP_CARRIER	OP_CARRIER_FL_NUM	ORIGIN	ORIGIN_CITY_NAME	DEST	DEST_CITY_NAME	CRS_DEP_TIME	DEP_TIME
1	null	DL	1451	BOS	Boston, MA	ATL	Atlanta, GA	1115	1113
2	null	DL	1479	BOS	Boston, MA	ATL	Atlanta, GA	1315	1311
3	null	DL	1857	BOS	Boston, MA	ATL	Atlanta, GA	1415	1414
4	null	DL	1997	BOS	Boston, MA	ATL	Atlanta, GA	1715	1720
5	null	DL	2065	BOS	Boston, MA	ATL	Atlanta, GA	2015	2010
6	null	US	2619	BOS	Boston, MA	ATL	Atlanta, GA	650	649
7	null	US	2621	BOS	Boston, MA	ATL	Atlanta, GA	1440	1446

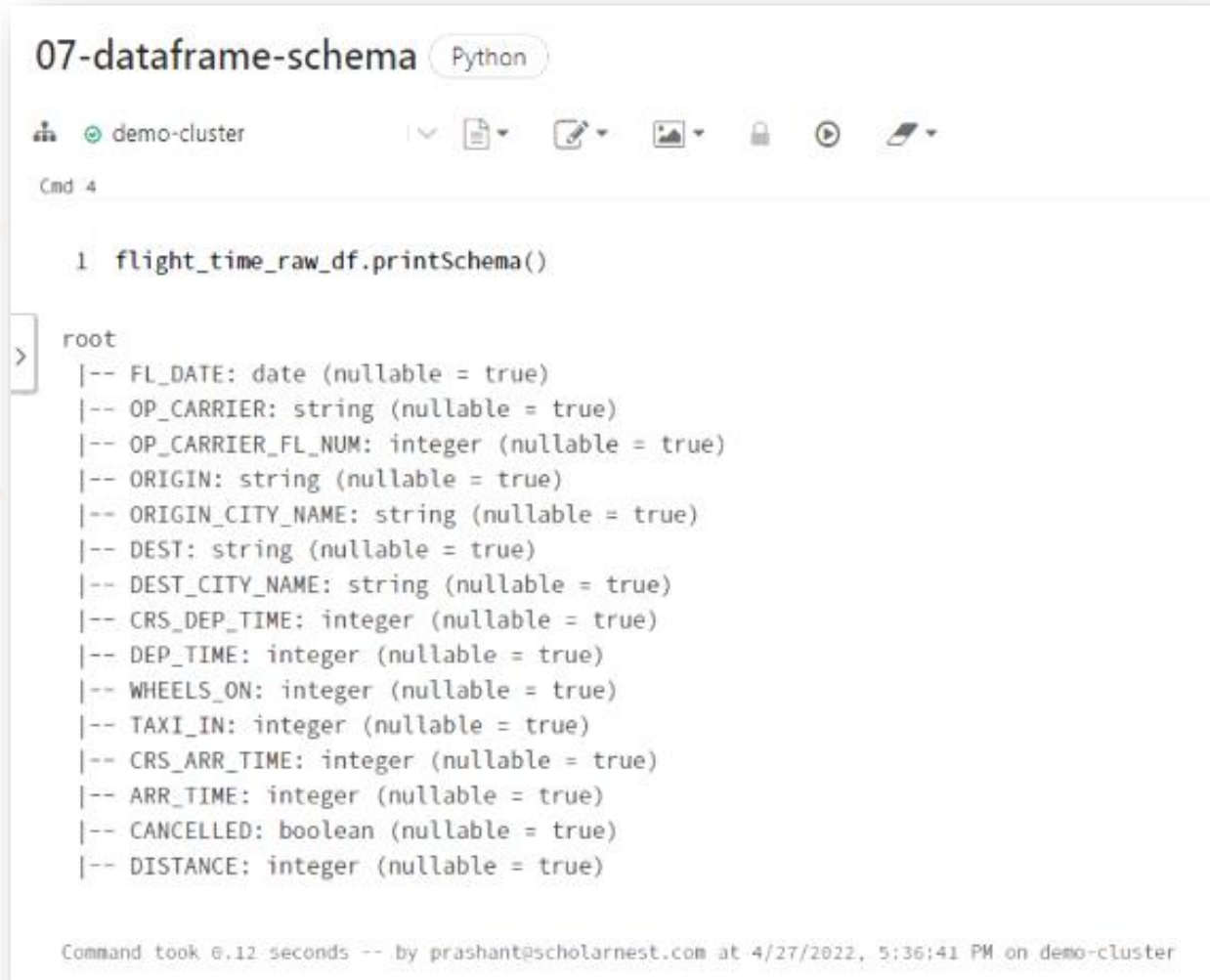
Truncated results, showing first 1000 rows.

Click to re-execute with maximum result limits.

Here is my schema definition shown below.

The FL_DATE is a date, and the CANCELLED column is a BOOLEAN.

The date, timestamp, and the boolean values are often loaded incorrectly, or they became null. You will see it almost everywhere.



```
07-dataframe-schema Python
demo-cluster
Cmd 4

1 flight_time_raw_df.printSchema()

root
|-- FL_DATE: date (nullable = true)
|-- OP_CARRIER: string (nullable = true)
|-- OP_CARRIER_FL_NUM: integer (nullable = true)
|-- ORIGIN: string (nullable = true)
|-- ORIGIN_CITY_NAME: string (nullable = true)
|-- DEST: string (nullable = true)
|-- DEST_CITY_NAME: string (nullable = true)
|-- CRS_DEP_TIME: integer (nullable = true)
|-- DEP_TIME: integer (nullable = true)
|-- WHEELS_ON: integer (nullable = true)
|-- TAXI_IN: integer (nullable = true)
|-- CRS_ARR_TIME: integer (nullable = true)
|-- ARR_TIME: integer (nullable = true)
|-- CANCELLED: boolean (nullable = true)
|-- DISTANCE: integer (nullable = true)

Command took 0.12 seconds -- by prashant@scholarnest.com at 4/27/2022, 5:36:41 PM on demo-cluster
```

If you check the requirements, you can see that FL_DATE is the flight date in MM/dd/yyyy format, and the CANCELLED is one or zero.

The default data format for DataFramereader is yyyy-MM-dd.
So the DataFramereader expected the FL_DATE to be in yyyy-MM-dd.

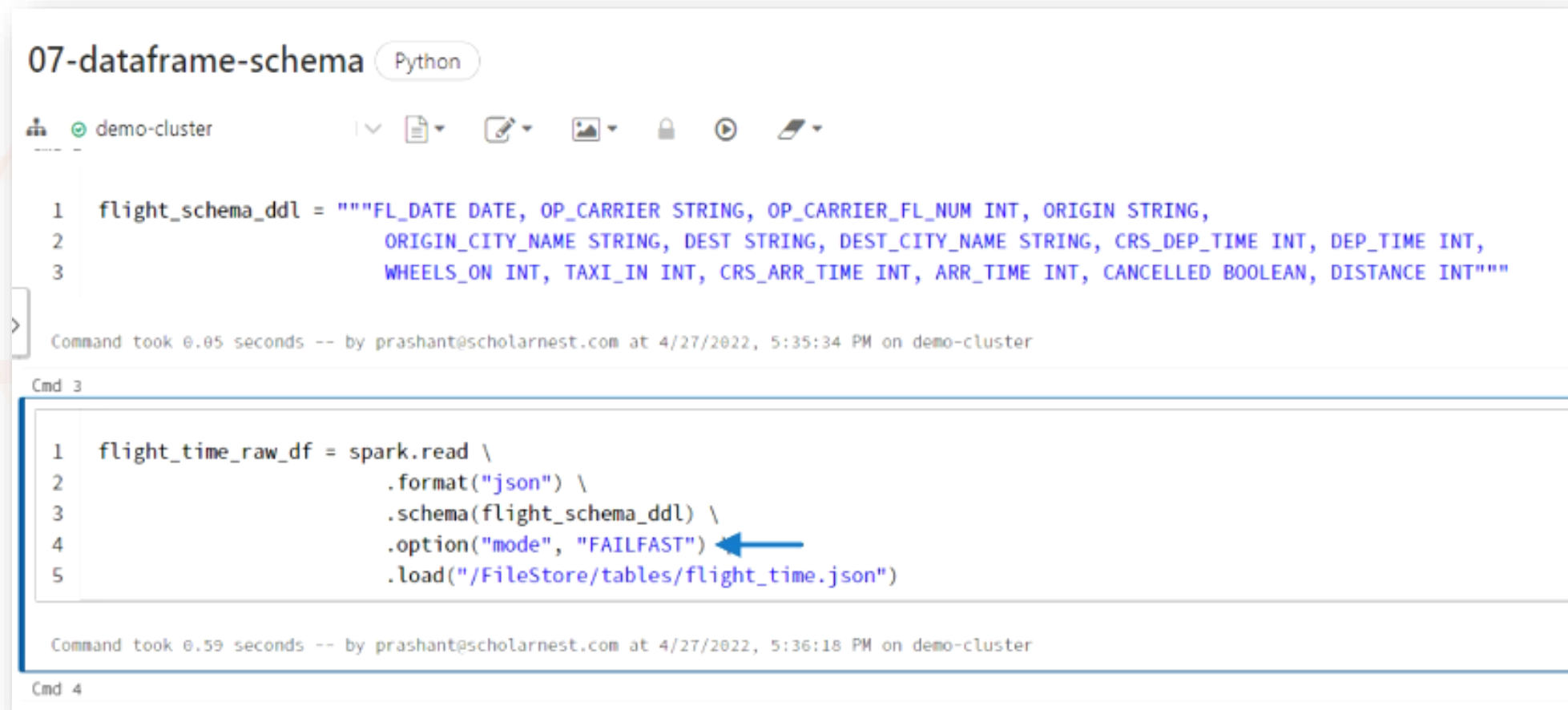
Similarly, the default format for a BOOLEAN is TRUE or FALSE.
So the DataFrameReader expected the FL_DATE to be TRUE or FALSE.

When the data is not found in the expected format, it fails the schema validation, and the Spark reads a null.

It doesn't throw an error. However, you can change this behaviour by setting the mode option.

So am setting the DataFrameReader mode to FAILFAST.

So now, if the DataFrameReader detects an incorrect value, it will stop and throw an exception.



The screenshot shows a Databricks notebook interface. The title bar indicates the notebook is named '07-dataframe-schema' and is running Python code. Below the title bar, there is a toolbar with various icons for file operations and execution. The main area of the notebook displays two code blocks. The first code block defines a schema for a flight table. The second code block shows the configuration of a DataFrameReader to use FAILFAST mode. A blue arrow points to the 'mode' option in the second code block. The notebook also shows the execution time and user for each command.

```
07-dataframe-schema Python

demo-cluster

1 flight_schema_ddl = """FL_DATE DATE, OP_CARRIER STRING, OP_CARRIER_FL_NUM INT, ORIGIN STRING,
2                      ORIGIN_CITY_NAME STRING, DEST STRING, DEST_CITY_NAME STRING, CRS_DEP_TIME INT, DEP_TIME INT,
3                      WHEELS_ON INT, TAXI_IN INT, CRS_ARR_TIME INT, ARR_TIME INT, CANCELLED BOOLEAN, DISTANCE INT"""

Command took 0.05 seconds -- by prashant@scholarnest.com at 4/27/2022, 5:35:34 PM on demo-cluster

Cmd 3

1 flight_time_raw_df = spark.read \
2                      .format("json") \
3                      .schema(flight_schema_ddl) \
4                      .option("mode", "FAILFAST")
5                      .load("/FileStore/tables/flight_time.json")

Command took 0.59 seconds -- by prashant@scholarnest.com at 4/27/2022, 5:36:18 PM on demo-cluster

Cmd 4
```

The DataFrameReader mode option can take three values.

1. FAIL FAST

The fail-fast will throw an error when detecting a malformed record.

2. DROPMALFORMED

The DROPMALFORMED will leave the malformed record and load others.

3. PERMISSIVE

The PERMISSIVE is the default value, and it will set the malformed column to null.

If you try running the display. You see an exception as shown below.

But how do we fix it?

Well, the error is coming because the data is not in the expected format.

We cannot change the file data. But we can change the expected format.



The screenshot shows a Databricks workspace interface. At the top, there's a header "07-dataframe-schema" with a "Python" language selector. Below this, a toolbar contains icons for cluster management, file operations, and other workspace functions. The main area displays the output of a command, which has been cancelled. The output shows a schema definition: |-- CANCELLED: boolean (nullable = true) and |-- DISTANCE: integer (nullable = true). A red vertical bar on the right indicates an error. Below the output, a message states "Command took 0.12 seconds -- by prashant@scholarnest.com at 4/27/2022, 5:36:41 PM on demo-cluster". A section labeled "Cmd 5" shows the command `display(flight_time_raw_df)`. Below the command, a section titled "(1) Spark Jobs" contains a detailed error message: `FileReadException: Error while reading file dbfs:/FileStore/tables/flight_time.json. Caused by: SparkException: Malformed records are detected in record parsing. Parse Mode: FAILFAST. To process malformed records as null result, try setting the option 'mode' as 'PERMISSIVE'. Caused by: BadRecordException: java.time.DateTimeException: Cannot cast 1/1/2000 to DateType. To return NULL instead, use 'try_cast'. If necessary set spark.sql.ansi.enabled to false to bypass this error. Caused by: DateTimeException: Cannot cast 1/1/2000 to DateType. To return NULL instead, use 'try_cast'. If necessary set spark.sql.ansi.enabled to false to bypass this error.` The error message is followed by another line: "Command took 1.33 seconds -- by prashant@scholarnest.com at 4/27/2022, 5:44:33 PM on demo-cluster". At the bottom, a section labeled "Cmd 6" is visible.

```
07-dataframe-schema Python

demo-cluster
|-- CANCELLED: boolean (nullable = true)
|-- DISTANCE: integer (nullable = true)

Command took 0.12 seconds -- by prashant@scholarnest.com at 4/27/2022, 5:36:41 PM on demo-cluster

Cmd 5
>
1 display(flight_time_raw_df)

(1) Spark Jobs
FileReadException: Error while reading file dbfs:/FileStore/tables/flight_time.json.
Caused by: SparkException: Malformed records are detected in record parsing. Parse Mode: FAILFAST. To process malformed records as null result, try setting the option 'mode' as 'PERMISSIVE'.
Caused by: BadRecordException: java.time.DateTimeException: Cannot cast 1/1/2000 to DateType. To return NULL instead, use 'try_cast'. If necessary set spark.sql.ansi.enabled to false to bypass this error.
Caused by: DateTimeException: Cannot cast 1/1/2000 to DateType. To return NULL instead, use 'try_cast'. If necessary set spark.sql.ansi.enabled to false to bypass this error.

Command took 1.33 seconds -- by prashant@scholarnest.com at 4/27/2022, 5:44:33 PM on demo-cluster

Cmd 6
```

I am adding another DataFrameReader option.

I am setting the expected date format to M/d/y because my file data is in the M/d/y format.

With this change, the FL_DATE column will load correctly.

Cmd 3

```
1 flight_time_raw_df = spark.read \  
2     .format("json") \  
3     .schema(flight_schema_ddl) \  
4     .option("mode", "FAILFAST") \  
5     .option("dateFormat", "M/d/y") ←   
6     .load("/FileStore/tables/flight_time.json")
```

Command took 0.59 seconds -- by prashant@scholarnest.com at 4/27/2022, 5:47:00 PM on demo-cluster

Unfortunately, the DataFrameReader does not allow us to change the expected Boolean format. So I do not have an option to load the zero/one as Boolean.

But that's not a problem, we have a straightforward approach. Set the columns to string and load it.

We can convert it to appropriate data types after loading.

So I have changed the CANCELLED column to a string so we can load it.

Cmd 2

```
1 flight_schema_ddl = """FL_DATE DATE, OP_CARRIER STRING, OP_CARRIER_FL_NUM INT, ORIGIN STRING,  
2 ORIGIN_CITY_NAME STRING, DEST STRING, DEST_CITY_NAME STRING, CRS_DEP_TIME INT, DEP_TIME INT,  
3 WHEELS_ON INT, TAXI_IN INT, CRS_ARR_TIME INT, ARR_TIME INT, CANCELLED STRING, DISTANCE INT"""
```

Command took 0.06 seconds -- by prashant@scholarnest.com at 4/27/2022, 5:47:40 PM on demo-cluster

Cmd 3

```
1 flight_time_raw_df = spark.read \  
2     .format("json") \  
3     .schema(flight_schema_ddl) \  
4     .option("mode", "FAILFAST") \  
5     .option("dateFormat", "M/d/y") \  
6     .load("/FileStore/tables/flight_time.json")
```

Command took 0.59 seconds -- by prashant@scholarnest.com at 4/27/2022, 5:47:00 PM on demo-cluster

Now if you try to check the Dataframe, you can see that it is loaded correctly as shown below.

Cmd 5

1 display(flight_time_raw_df)

(1) Spark Jobs

	FL_DATE	OP_CARRIER	OP_CARRIER_FL_NUM	ORIGIN	ORIGIN_CITY_NAME	DEST	DEST_CITY_NAME	CRS_DEP_TIME	DEP_TIME
1	2000-01-01	DL	1451	BOS	Boston, MA	ATL	Atlanta, GA	1115	1113
2	2000-01-01	DL	1479	BOS	Boston, MA	ATL	Atlanta, GA	1315	1311
3	2000-01-01	DL	1857	BOS	Boston, MA	ATL	Atlanta, GA	1415	1414
4	2000-01-01	DL	1997	BOS	Boston, MA	ATL	Atlanta, GA	1715	1720
5	2000-01-01	DL	2065	BOS	Boston, MA	ATL	Atlanta, GA	2015	2010
6	2000-01-01	US	2619	BOS	Boston, MA	ATL	Atlanta, GA	650	649
7	2000-01-01	US	2621	BOS	Boston, MA	ATL	Atlanta, GA	1440	1446

Truncated results, showing first 1000 rows.
[Click to re-execute with maximum result limits.](#)



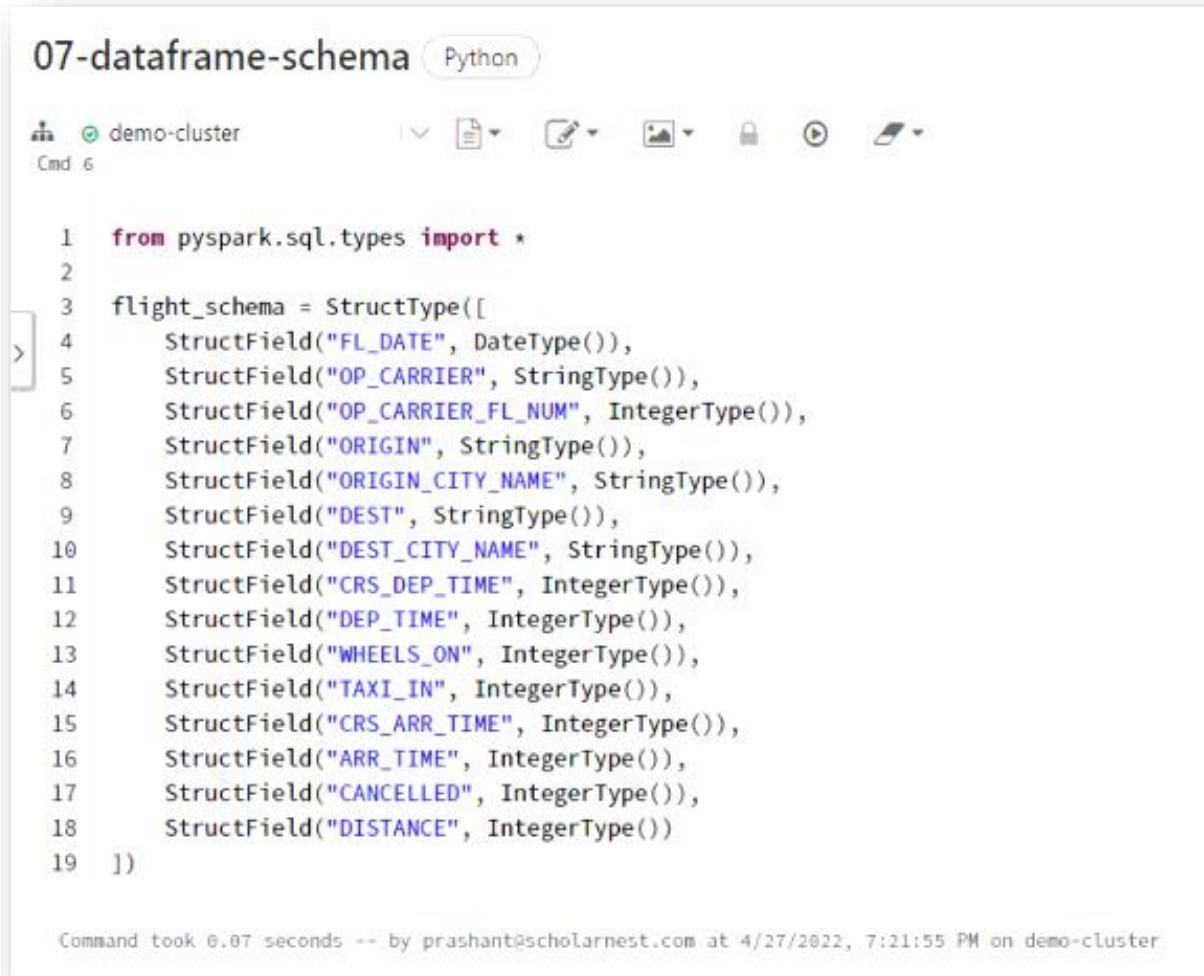
We have two approaches to defining a Schema.

1. Schema DDL String
2. StructType Object

We saw the first approach.

Now let's define the schema using the StructType object definition.

We have to import Spark types from the `pyspark.sql.types` package. Then I can define a variable and set it to `StructType()`. The `StructType()` method takes a list of `StructField()`. You can put a comma and add all the available fields as shown below.



```
07-dataframe-schema Python
demo-cluster
Cmd 6

1 from pyspark.sql.types import *
2
3 flight_schema = StructType([
4     StructField("FL_DATE", DateType()),
5     StructField("OP_CARRIER", StringType()),
6     StructField("OP_CARRIER_FL_NUM", IntegerType()),
7     StructField("ORIGIN", StringType()),
8     StructField("ORIGIN_CITY_NAME", StringType()),
9     StructField("DEST", StringType()),
10    StructField("DEST_CITY_NAME", StringType()),
11    StructField("CRS_DEP_TIME", IntegerType()),
12    StructField("DEP_TIME", IntegerType()),
13    StructField("WHEELS_ON", IntegerType()),
14    StructField("TAXI_IN", IntegerType()),
15    StructField("CRS_ARR_TIME", IntegerType()),
16    StructField("ARR_TIME", IntegerType()),
17    StructField("CANCELLED", IntegerType()),
18    StructField("DISTANCE", IntegerType())
19 ])

Command took 0.07 seconds -- by prashant@scholarnest.com at 4/27/2022, 7:21:55 PM on demo-cluster
```

We use the StructType object in the same way as we used the DDL schema. Here is the code for the same.

So, you can define the schema using a DDL string or using StructType.

Cmd 7

```
1 flight_time_raw_df_1 = spark.read \  
2     .format("json") \  
3     .schema(flight_schema) \  
4     .option("mode", "FAILFAST") \  
5     .option("dateFormat", "M/d/y") \  
6     .load("/FileStore/tables/flight_time.json")
```

Command took 0.51 seconds -- by prashant@scholarnest.com at 4/27/2022, 7:23:59 PM on demo-cluster



Thank You
ScholarNest Technologies Pvt Ltd.
www.scholarnest.com